

# **JOB PORTAL SYTEM**

## **A PROJECT REPORT**

*Submitted by*

Vishwam Mishra-24BCS10087

*in partial fulfillment for the award of the degree of*

**BACHELOR OF ENGINEERING**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**



## INTRODUCTION

The rapid growth of digital platforms has significantly transformed the recruitment process, enabling faster and more efficient interaction between employers and job seekers. Traditional hiring methods, which relied heavily on manual processes and limited reach, are increasingly being replaced by web-based job portal systems that provide centralized access to employment opportunities.

This project, Job Portal System, is a full-stack web application designed to streamline the process of job posting and job application. It enables administrators to post job listings and manage opportunities, while allowing users to browse available jobs and apply seamlessly through an intuitive interface. The system aims to bridge the gap between recruiters and candidates by offering a simple, accessible, and user-friendly platform.

The application is built using modern technologies, with a React-based frontend for dynamic user interaction and a Spring Boot backend for handling business logic and data persistence. Data is stored using a relational database (H2), ensuring structured and efficient management of job listings and applications. Communication between frontend and backend is achieved through RESTful APIs.

## OBJECTIVES

The main objectives of this project are:

- To develop a **basic full-stack web application**
- To understand **client-server architecture**
- To implement **REST APIs using Express.js**
- To build a dynamic user interface using **React**
- To implement **search and filtering functionality**
- To simulate real-world job portal behavior
- To create a project that is **easy to explain in viva**

## FUNCTIONAL REQUIREMENTS :

### User Functional Requirements

#### 1. View Job Listings

- The system should display all available jobs.
- Each job should be shown as a **card** containing:
  - Job title
  - Company name
  - Location
  - Category

#### 2. Search Jobs

- User should be able to search jobs using keywords.
- Search should work on:
  - Job title
  - Company name
- The system filters results dynamically.

### **3. Filter Jobs by Category**

- Users should be able to filter jobs based on categories such as:
  - IT
  - Marketing
  - Finance
- Only matching jobs should be displayed.

### **4. View Job Details**

- User should be able to click on a job and view:
  - Full description
  - Requirements
  - Salary
  - Location

### **5. Apply for Job**

- User should be able to fill an application form.
- Form fields:
  - Name
  - Email
  - Phone
  - Resume (text input)

### **6. Application Submission**

- After submission:
  - System should validate input
  - Send data to backend
  - Show success message

## **System Functional Requirements**

### **Job Management (Admin Side)**

- The system shall allow the administrator to **create (post) new job listings**.
- The system shall accept job details including:
  - Job Title
  - Company Name
  - Location
  - Job Description
- The system shall store the job data in the database.
- The system shall allow retrieval of all posted jobs.

---

### **Job Browsing (User Side)**

- The system shall display a list of all available jobs.
- The system shall present job details such as:
  - Title

- Company
    - Location
    - Description
  - The system shall allow users to **search jobs** based on keywords (title, company, location, description).
  - The system shall dynamically update the job list based on search input.
- 

### **Job Application**

- The system shall allow users to **apply for a selected job**.
  - The system shall collect applicant details:
    - Name
    - Email
  - The system shall associate the application with a specific job using **job ID**.
  - The system shall store application data in the database.
  - The system shall provide feedback (e.g., confirmation message) after successful submission.
- 

### **User Interface Interaction**

- The system shall provide navigation between:
    - Landing Page
    - Browse Jobs Page
    - Admin Page
  - The system shall display forms for input and validate basic user input.
  - The system shall use modal dialogs for job applications.
- 

### **Backend Services**

- The system shall expose REST APIs for:
    - Fetching all jobs (GET /jobs)
    - Adding a job (POST /jobs)
    - Submitting an application (POST /applications)
  - The system shall process incoming requests and return appropriate responses.
- 

### **Data Storage**

- The system shall store job and application data in a database.
  - The system shall ensure each job and application has a unique identifier.
- 

### **Error Handling**

- The system shall handle invalid requests and display appropriate error messages.
- The system shall prevent application submission if required fields are empty (basic validation).

## **4. DATA FLOW DIAGRAM (DFD)**

### **Level 0**

User → Frontend → Backend → Data Storage

### **Level 1**

User Input → React UI → API Request → Express Server → jobs.js → Response → UI Update

## **5. SYSTEM ARCHITECTURE**

The system follows **Client-Server Architecture**:

### **Frontend (Client)**

- React.js
- Handles UI and user interaction

### **Backend (Server)**

- Node.js + Express
- Handles API and data processing

### **Data Layer**

- JavaScript array (jobs.js)

## **6. SYSTEM LAYOUT**

### **Pages Structure**

Home → Jobs → Job Details → Apply

### **UI Components**

- Navbar
- Job Cards
- Search Bar
- Filter Dropdown
- Footer

## **7. LIMITATIONS**

### **1. No Database**

- Data is not stored permanently
- All jobs are hardcoded

### **2. No Authentication**

- No login/signup system
- Anyone can apply

### **3. No Data Persistence**

- Applications are not saved
- Data is lost on server restart

### **4. No Resume Upload**

- Resume is only text input
- No file handling

### **5. Limited Scalability**

- Cannot handle large number of users
- Not suitable for real-world deployment

### **6. No Admin Panel**

- Cannot add/edit/delete jobs dynamically

## **8. FUTURE ENHANCEMENTS**

- Add MongoDB database
- Implement login system
- Add admin dashboard
- Enable resume upload
- Improve UI/UX
- Deploy application online

## 9. CONCLUSION

The Job Portal System is a simple yet effective full-stack application that demonstrates how frontend and backend interact in a real-world system.

The project successfully implements:

- API communication
- Dynamic UI updates
- Form handling
- Search and filtering logic

Although it has limitations, it serves as a strong foundation for understanding full-stack development concepts.